Studio Nx

백엔드 184 — save_sample()

2026-10-15

Backend(FastAPI/Python) 184 — `save_sample()`

항목	내용
파일	`studionx/backend/app/services/sample_service.py` (새 파일)
작업	S1-07 샘플 라이브러리: 업로드 저장 — 파일 저장, 메타·파형 생성, samples 행, 사용량 증가 (FR-221·131)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	183 `check_upload()` · 185 용량 재계산 (예정)

1. 하는 일

올린 파일을 `storage/samples/{사용자 번호}/{무작위 32자}.{확장자}` 에 저장하고, 같은 트랜잭션에서 `samples` 행을 넣고 `users.storage_used_bytes`를 파일 크기만큼 늘립니다. 완성된 `Sample`을 돌려줍니다.

항목	값
`file_path`	`samples/{owner_id}/{uuid4().hex}{확장자}` (확장자는 소문자)
`waveform_path`	`write_waveform`(182)가 만든 `waveforms/{owner_id}/{uuid}.json`
`name`	올린 파일 이름에서 확장자를 뗀 값 (`kick.wav` → `kick`), 200자까지
`original_filename`	올린 파일 이름 그대로, 255자까지
`mime_type`	검사를 통과한 확장자로 정합니다 (`audio/wav`, `audio/mpeg`, …). 브라우저가 알려 준 값은 믿지 않습니다
`duration_ms` · `sample_rate` · `channels`	`read_audio_info`(180)
`source`	`SampleSource.UPLOAD`
`users.storage_used_bytes`	파일 크기만큼 증가 (FR-131)

2. 언제 쓰나

업로드 흐름의 마지막 단계입니다: `check_upload`(183) → **`save_sample`(184)** → `read_audio_info`(180) → `compute_waveform`(181) → `write_waveform`(182) → `samples` 행. 라우터(`POST /samples`)가 이 함수를 부릅니다.

저장 경로 상수는 `SAMPLE_DIR = "samples"`, 형식별 MIME은 `MIME_TYPES`, 나눠 읽는 크기는 `CHUNK_BYTES`(1MB)입니다.

3. 코드

상수 (파일 위쪽)

```
SAMPLE_DIR = "samples"
MIME_TYPES = {
    ".wav": "audio/wav",
    ".mp3": "audio/mpeg",
    ".flac": "audio/flac",
    ".ogg": "audio/ogg",
```

```
    ".aiff": "audio/aiff",
    ".aif": "audio/aiff",
}
CHUNK_BYTES = 1024 * 1024
```

함수 (뼈대)

```
async def save_sample(db: Session, user: User, file: UploadFile) -> Sample:
    settings = get_settings()
    storage_dir = settings.storage_dir
    original_filename = file.filename or ""
    extension = Path(original_filename).suffix.lower()
    relative = f"{SAMPLE_DIR}/{user.user_id}/{uuid4().hex}{extension}"
    target = storage_dir / relative
    target.parent.mkdir(parents=True, exist_ok=True)

    file_size = 0
    with target.open("wb") as out:
        while True:
            chunk = await file.read(CHUNK_BYTES)
            if not chunk:
                break
            file_size += len(chunk)
            out.write(chunk)
            if file_size > settings.sample_max_bytes:
                break

    waveform_relative: str | None = None
    try:
        check_upload(user, original_filename, file_size)
        info = read_audio_info(target)
        waveform = compute_waveform(target)
        waveform_relative = write_waveform(waveform, owner_id=user.user_id)

        stem = Path(original_filename).stem.strip()
        sample = Sample(
            owner_id=user.user_id,
            name=stem[:200] or "Untitled",
            original_filename=original_filename[:255],
            file_path=relative,
            mime_type=MIME_TYPES[extension],
            file_size=file_size,
            duration_ms=info.duration_ms,
            sample_rate=info.sample_rate,
            channels=info.channels,
            waveform_path=waveform_relative,
            source=SampleSource.UPLOAD,
        )
        db.add(sample)
        user.storage_used_bytes = user.storage_used_bytes + file_size
        db.commit()
    except Exception:
        db.rollback()
        target.unlink(missing_ok=True)
        if waveform_relative:
            (storage_dir / waveform_relative).unlink(missing_ok=True)
```

```
        raise
    db.refresh(sample)
    return sample
```

문법 풀이

식	뜻
`await file.read(CHUNK_BYTES)`	올린 파일에서 1MB씩 읽습니다 (한 번에 100MB를 메모리에 올리지 않습니다)
`with target.open("wb") as out`	저장할 파일을 열어 두고 블록을 벗어날 때 닫습니다
`if file_size > settings.sample_max_bytes: break`	한도를 넘으면 더 읽지 않습니다 (디스크를 다 채우지 않습니다)
`except Exception:`	아래 어느 단계에서든 오류가 나면 파일을 지우고 되돌립니다
`target.unlink(missing_ok=True)`	파일이 이미 없어도 오류 없이 지웁니다
`MIME_TYPES[extension]`	183을 통과한 확장자라 반드시 표에 있습니다

4. 흐름

1. 저장할 경로(`samples/{사용자}/{무작위}.{확장자}`)를 만들고 폴더를 준비합니다.
2. 올린 파일을 1MB씩 나눠 쓰면서 크기를 셉니다. 한도를 넘으면 읽기를 멈춥니다.
3. `check_upload`(183)로 형식·크기·남은 용량을 확인합니다. 걸리면 2에서 쓴 파일을 지우고 그 오류를 그대로 올립니다.
4. `read_audio_info`(180)로 메타를, `compute_waveform`(181)과 `write_waveform`(182)로 파형 파일을 만듭니다. 못 읽는 파일이면 415로 멈추고 쓴 파일을 지웁니다.
5. `samples` 행을 넣고 `user.storage_used_bytes`를 파일 크기만큼 늘려 **한 번의 commit**으로 저장합니다.
6. commit이 실패하면 rollback으로 되돌리고, 남은 파일 둘(샘플·파형)을 지웁니다.

5. 왜 이렇게

선택	이유
파일 저장과 행·사용량을 한 함수에	183이 읽는 사용량을 올리는 곳이 여기뿐입니다. 나누면 `samples` 행만 생기고 사용량이 안 늘어 FR-135(한도 차단)가 동작하지 않습니다
한 번의 commit	행과 사용량이 따로 저장돼 어긋난 상태가 남지 않습니다 (사용자와 합의, 2026-10-15)
오류 시 파일까지 지움	DB는 되돌리지만 디스크 파일은 자동으로 안 지워집니다. 반쪽 상태를 남기지 않으려고 직접 지웁니다
형식 검사를 파일을 쓴 뒤에	100MB를 메모리에 올리지 않으려고 나눠 쓰기 때문입니다. 어차피 걸리면 지우므로 남는 파일이 없습니다
MIME을 확장자로 정함	브라우저가 보내는 `content_type`은 조작될 수 있습니다. 검사를 통과한 확장자를 씁니다 (프로필 사진 38과 같은 방침)
`name`은 파일 이름에서 확장자를 뗌	라이브러리 목록에 보여 줄 이름입니다. 사용자가 따로 정하기 전까지의 기본값입니다
이름·파일 이름 길이 제한	DB 칸(`name` 200자, `original_filename` 255자)에 맞춰 자릅니다

6. 시험 (`tests/test_sample_service.py` 새 파일 12개, 전체 929개 통과)

메모리에서 오디오를 만들어 올리고(`BytesIO`), 저장 위치는 임시 폴더로 바꿉니다. 행과 사용량은 실제 MySQL로 확인합니다.

시험	확인한 것
저장·행 생성	경로 모양, 파일 내용 일치, `name`·`original_filename`·`mime_type`·`file_size`·`duration_ms`·`sample_rate`·`channels`·`source`·`deleted_at`, `sample_id`
사용량 증가	`user.storage_used_bytes`와 DB 값이 파일 크기만큼 늘어남
파형 파일	경로 모양과 JSON 내용(`sample_rate`, `channels`, peaks 1000칸)
FLAC	확장자 `.flac`, `mime_type` `audio/flac`
대소문자	`LOOP.WAV`도 `.wav`, `audio/wav`
확장자 거부	`.txt`면 415, 파일·행·사용량이 남지 않음
못 읽는 파일	이름만 `.wav`인 텍스트는 415(180), 파일·행이 남지 않음
크기 초과	413 `FILE_TOO_LARGE`, 파일·행이 남지 않음
용량 초과	413 `STORAGE_QUOTA_EXCEEDED`, 사용량은 그대로
commit 실패	rollback으로 행이 남지 않고 샘플·파형 파일을 지움
두 번 올리기	파일 이름·파형 이름이 다르고 사용량이 합계만큼 늘어남
긴 파일 이름	`name` 200자, `original_filename` 255자로 잘림